

F1 Database Design — v2 (PostgreSQL, hybrid relational + JSONB)

Updated normalized (3NF) schema bridging Jolpica/Ergast (historical, 1950+) and OpenF1 (telemetry, 2023+). Domain color key: reference = teal, event = blue, telemetry = coral, transient live = amber, prediction = purple. This revision adds a season-roster table, a transient live-timing table, and a JSONB-based predictions model.

What changed in v2

#	Change	Why
1	Added season_entries (reference)	Driver→team is a function of season, not of the driver. Replaces the idea of a constructor_id column on drivers; correct for every historical season and available before a race runs (predictor needs it).
2	Added live_timing (transient)	Lap-by-lap running order — every driver's position at the end of each lap — plus gap / interval / last-lap, populated from OpenF1 during a race, consumed by the model every ~10 laps, then truncated once official results are ingested.
3	Redesigned predictions to JSONB	One row per (session, checkpoint) with promoted scalar columns + a payload JSONB holding the full ranked grid. This is the document-model flexibility you wanted — inside Postgres.
4	Circuit maps + assets (frontend only)	GeoJSON static assets as the primary path; OpenF1 /location GPS as the dynamic phase-2 path. No backend tables required now; one optional nullable column noted for later.

Architecture decision — MongoDB vs PostgreSQL

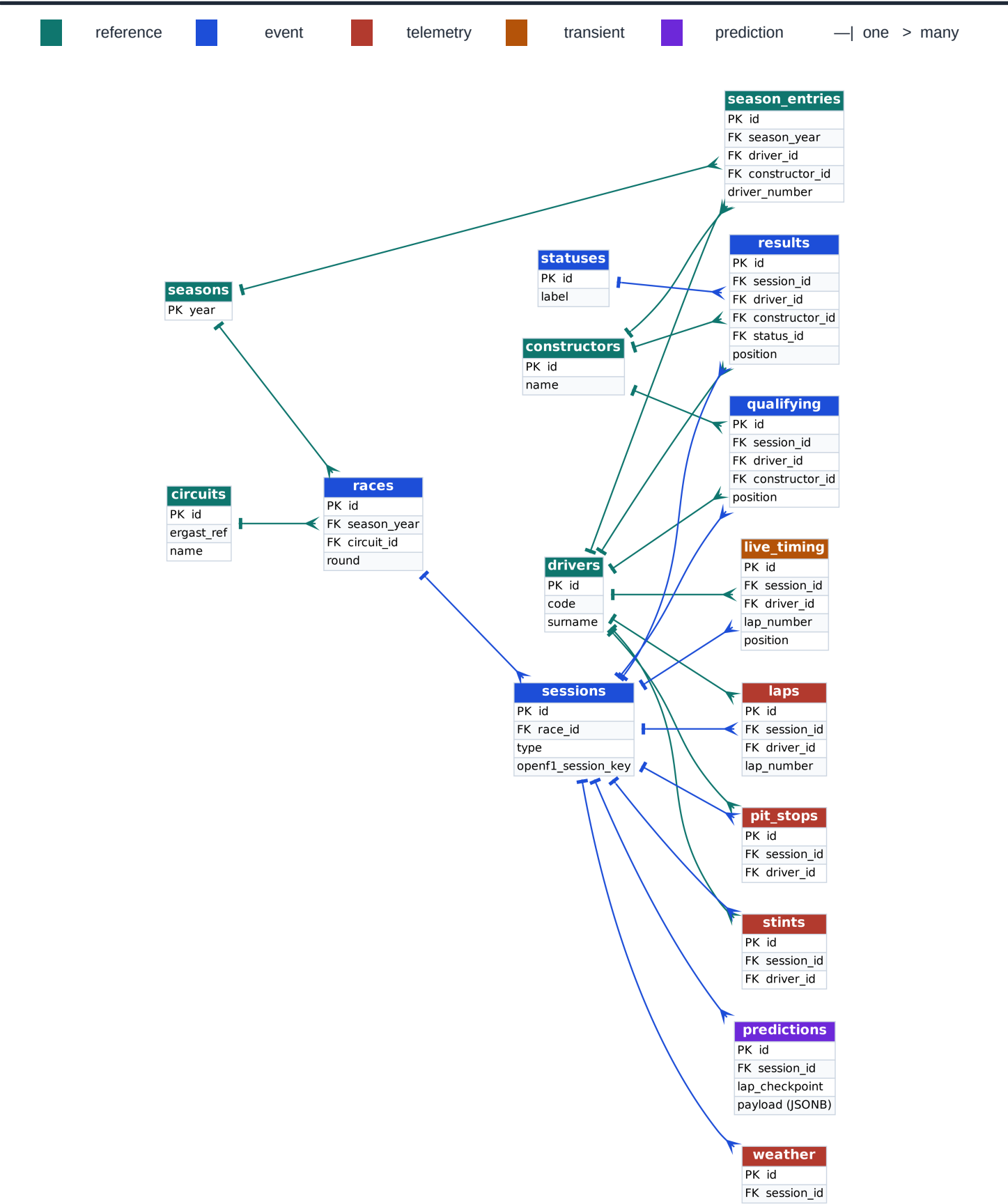
Verdict: stay on PostgreSQL; adopt a hybrid relational + JSONB model. Do not move to MongoDB. MongoDB is technically *feasible* here — it could store this data — but feasible is not advisable. The reasoning, as an architect:

Dimension	Winner	Rationale
Workload shape	PostgreSQL	The product is historical analysis + prediction: cross-season aggregations and multi-way joins across drivers, constructors, results, laps, stints. That is the relational/analytical sweet spot. Mongo's \$lookup is weaker and more awkward for multi-collection joins.
Referential integrity	PostgreSQL	drivers, constructors, sessions and statuses are referenced by many tables. Postgres enforces these foreign keys; in Mongo you hand-roll integrity in application code and inevitably drift.
The 'flexible' bits	Postgres JSONB	The two places you reached for Mongo — transient live state and JSON predictions — are exactly where JSONB excels. You get document flexibility without a second datastore.
Cost & ops (\$20/mo)	PostgreSQL	You already run Neon (free Postgres) + Render. Adding Mongo Atlas means two datastores, two mental models, and a split between your relational core and a document store. One engine is cheaper and simpler.
Portfolio signal	PostgreSQL	For Apple/Google, demonstrating clean relational modeling plus a deliberate JSONB hybrid — and being able to defend the choice — reads as more senior than a Mongo rewrite for data that is plainly relational.

One-line takeaway: the JSONB columns below give you everything MongoDB was tempting you with, while keeping foreign keys, SQL joins, one bill, and one database to operate.

Schema overview — entity-relationship diagram

Every table, its primary key (PK) and foreign keys (FK), and how they connect. Read a bar as the “one” side and a crow's foot as the “many” side of each link.



Reference tables

seasons

One row per championship year.

year
2023
2024
2025

circuits

Static track info that never changes (the live-race anchor).

id	ergast_ref	name	locality	country	lat	lng	alt
1	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.42056	7
2	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.01694	153

drivers

One row per driver; ergast_ref + permanent_number bridge the two APIs. **No constructor_id here** — team membership lives in season_entries (below).

id	ergast_ref	permanent_number	code	forename	surname	dob	nationality
1	max_verstappen	1	VER	Max	Verstappen	1997-09-30	Dutch
2	leclerc	16	LEC	Charles	Leclerc	1997-10-16	Monegasque
3	hamilton	44	HAM	Lewis	Hamilton	1985-01-07	British

constructors

One row per team; rebrands are new rows. color / logo_url are supplied by us (APIs lack historical branding).

id	ergast_ref	name	nationality	color	logo_url
1	red_bull	Red Bull	Austrian	#3671C6	/logos/red_bull.png
2	ferrari	Ferrari	Italian	#E8002D	/logos/ferrari.png
3	mercedes	Mercedes	German	#27F4D2	/logos/mercedes.png

season_entries (NEW)

The canonical season roster: one row per (season, driver) with the team that driver races for that year, plus that season's car number. Unique on (season_year, driver_id). This is the correct answer to “which driver is on which team” — it is right for every historical season and exists *before* any session result, which the predictor depends on. Note the Hamilton rows: Mercedes in 2024, Ferrari in 2025. A single constructor_id column on drivers could not represent that without silently corrupting historical joins. For race-by-race truth (incl. mid-season swaps), results.constructor_id remains the per-session source of record.

id	season_year (FK)	driver_id (FK)	constructor_id (FK)	driver_number
1	2024	1 (Verstappen)	1 (Red Bull)	1
2	2024	2 (Leclerc)	2 (Ferrari)	16
3	2024	3 (Hamilton)	3 (Mercedes)	44
4	2025	3 (Hamilton)	2 (Ferrari)	44
5	2025	1 (Verstappen)	1 (Red Bull)	1

Event tables

races

One Grand Prix weekend. Unique on (season_year, round).

id	season_year	round	circuit_id	name	race_date	race_time
1	2024	8	1	Monaco Grand Prix	2024-05-26	13:00:00
2	2024	12	2	British Grand Prix	2024-07-07	14:00:00

sessions

Each session in a weekend (fp1-3, qualifying, sprint_qualifying, sprint, race); openf1_session_key links OpenF1 telemetry.

id	race_id	type	scheduled_start	openf1_session_key
1	1	race	2024-05-26 13:00	9523
2	1	qualifying	2024-05-25 15:00	9520
3	1	fp1	2024-05-24 13:30	9518

statuses

Normalized finish status, referenced by results (no repeated strings).

id	label
1	Finished
5	Engine
11	+1 Lap

results

Per-driver result for a session (race or sprint). Fed only by the official final classification, never by live data. constructor_id here is the per-session source of truth for driver→team.

id	session_id	driver_id	constructor_id	grid	position	points	laps_completed	status_id	fastest_lap_rank
1	1	2	2	1	1	25	78	1	3
2	1	1	1	6	6	8	78	1	1

qualifying

Per-driver qualifying with Q1/Q2/Q3 as separate columns (1NF); tied to a session so sprint-qualifying is covered.

id	session_id	driver_id	constructor_id	position	q1	q2	q3
1	2	2	2	1	1:11.388	1:10.952	1:10.270
2	2	1	1	6	1:11.834	1:11.302	1:10.567

Telemetry tables (OpenF1, 2023+)

These are **permanent**. stints and pit_stops are static per session, so they are ingested straight into these tables once known — not treated as transient.

laps

One row per driver per lap.

id	session_id	driver_id	lap_number	lap_duration	sector1_ms	sector2_ms	sector3_ms	is_pit_out_lap
1	1	2	1	75.123	24500	28100	22523	true
2	1	2	2	73.880	23900	27800	22180	false

pit_stops

One row per pit stop. Static per session → permanent.

id	session_id	driver_id	lap_number	stop_number	duration
1	1	1	25	1	23.4
2	1	2	30	1	22.1

stints

Tyre stints with compound and lap range. Static per session → permanent.

id	session_id	driver_id	stint_number	compound	lap_start	lap_end
1	1	2	1	MEDIUM	1	28
2	1	2	2	HARD	29	78

weather

Time-stamped session weather readings.

id	session_id	recorded_at	air_temp	track_temp	humidity	pressure	rainfall	wind_speed	wind_dir
1	1	2024-05-26 13:05	24.5	48.2	52	1014.2	0	2.1	210

Transient live-timing table (NEW)

Working set for in-race state that is *not* in results until the race ends. **Grain: one row per driver per lap** — the complete running order at the end of every lap — with gap to leader, interval to the car ahead, and that lap's time. So each lap contributes one row per car on track (a 20-car grid → ~20 rows per lap). The predictor reads the trailing ~10-lap window at each checkpoint; rows for a session are deleted once that session's official results are ingested.

live_timing

Example: laps 10 and 11 of the Monaco race (3 of 20 cars shown).

id	session_id (FK)	driver_id (FK)	lap_number	position	gap_to_leader_ms	interval_ahead_ms	last_lap_ms	captured_at
1	1	2	10	1	0	0	75123	2024-05-26 13:24:58
2	1	1	10	2	2310	2310	75488	2024-05-26 13:24:58
3	1	3	10	3	5104	2794	75902	2024-05-26 13:24:58
4	1	2	11	1	0	0	74980	2024-05-26 13:26:13
5	1	1	11	2	2655	2655	75201	2024-05-26 13:26:13
6	1	3	11	3	5380	2725	75640	2024-05-26 13:26:13

Lifecycle: OpenF1 stream → INSERT into live_timing → model reads last ~10 laps → write predictions → on results ingest, DELETE FROM live_timing WHERE session_id = \$1.

Neon free-tier caveat (verified): on Neon's free plan, scale-to-zero cannot be disabled and UNLOGGED tables are truncated on every compute suspend — not just on crash as in standard Postgres. So live_timing is specified as a normal (logged) table with explicit DELETE cleanup. UNLOGGED is a valid write-throughput optimization only on a paid plan where scale-to-zero is off; either way the data is disposable (re-fetchable from OpenF1), so correctness is preserved.

Prediction table — JSONB redesign

Old shape: one row per (session, driver, checkpoint) with predicted_position + probability — N rows per checkpoint, and the model's richer output (per-driver win/podium probabilities, the feature snapshot it acted on) had nowhere to live.

New shape: one row per (session, checkpoint). Columns you filter, join, or score on are promoted to real columns; the full ranked grid lives in a payload JSONB. Predictions are kept permanently (so predicted vs actual can be scored later). A GIN index on payload supports querying inside it; for per-driver scoring you unnest with `jsonb_array_elements(payload->'grid')`.

predictions

Promoted scalar columns + JSONB payload.

id	session_id (FK)	lap_checkpoint	model_version	created_at	payload (JSONB) — see below
1	1	10	lgbm_v1	2024-05-26 13:25:00	{ grid: [...], features_snapshot: {...} }
2	1	20	lgbm_v1	2024-05-26 13:38:00	{ grid: [...], features_snapshot: {...} }

Example payload (row id = 1)

```
{
  "model_version": "lgbm_v1",
  "lap_checkpoint": 10,
  "generated_at": "2024-05-26T13:25:00Z",
  "grid": [
    { "driver_id": 2, "code": "LEC", "predicted_position": 1,
      "win_probability": 0.62, "podium_probability": 0.94 },
    { "driver_id": 1, "code": "VER", "predicted_position": 2,
      "win_probability": 0.21, "podium_probability": 0.71 },
    { "driver_id": 3, "code": "HAM", "predicted_position": 5,
      "win_probability": 0.03, "podium_probability": 0.34 }
  ],
  "features_snapshot": {
    "safety_car_active": false,
    "track_temp_c": 48.2,
    "leader_tyre": "MEDIUM",
    "leader_tyre_age_laps": 10
  }
}
```

Why this beats both the old row-per-driver design and MongoDB: the frontend gets the entire checkpoint in one row it can render directly; the model can evolve the payload shape without a migration; and you still have first-class SQL columns for the things you actually query — all with foreign keys intact and no second datastore.

Circuit maps & visual assets (front-end)

All of this is front-end only; no backend tables are required now. Recommended order of adoption:

Approach	Source	Notes
A. Static GeoJSON (recommended start)	bacinger/f1-circuits (GitHub)	Every F1 circuit, 1950→today, as GeoJSON LineStrings. Free, static. Drop the files in the frontend /public folder and render with D3, MapLibre, Leaflet, or raw SVG. Zero backend, no rate limits, works offline, version-controlled in your repo. Best fit for “maintain manually, portfolio-quality, front-end only.”
B. Dynamic GPS (phase 2)	OpenF1 /location endpoint	Real car GPS x/y/z at ~3.7 Hz. Plot one full lap of one car to derive a true track outline — and the same data later animates live car dots moving around the circuit, which is a strong portfolio feature. More work: needs smoothing / interpolation. Origin (0,0) is arbitrary, so normalize before drawing.
C. Corner annotations	MultiViewer API	api.multiviewer.app/api/v1/circuits/{key}/{year} returns corner numbers, marshal sectors, and circuit rotation — the same source FastF1 uses. Good for labeling corners on top of A or B.
D. Quick thumbnails	OpenF1 meetings	Exposes <code>circuit_image</code> (official F1 track-icon PNGs) and country flags — handy low-effort thumbnails for list views.

Recommendation: ship A now (static GeoJSON), then layer in B when you build the live race view. **Optional future column** (not needed yet): a nullable `circuits.geojson_path` if you ever want to resolve circuit→asset server-side.

Driver photos / team logos (deferred): [constructors.logo_url](#) already exists; [OpenF1's drivers endpoint](#) exposes `headshot_url` for driver photos. Revisit when you build the roster pages.

Ingestion & lifecycle rules

1. **results** is fed only by the official final classification (post-race penalties / DSQs change finishing order after the flag), never by live data.
2. **season_entries** is seeded per season from the entry list and is the roster the predictor reads before lights-out; per-session team truth still comes from `results.constructor_id`.
3. **stints** and **pit_stops** are static per session and ingested directly into their permanent telemetry tables.
4. **live_timing** is transient: streamed in during the race, read by the model every ~10 laps, then DELETED for that session once official results are ingested. Never promoted into results.
5. **predictions** are written every ~10 laps and kept permanently, so predicted vs actual can be scored later.

Key relationships (plain language)

The full entity-relationship diagram is on page 2. How to read it: a bar (—|) marks the “one” side of a relationship and a crow's foot (>) marks the “many” side. In words:

- One **season** has many **races**; one **circuit** hosts many **races**.
- One **race** weekend has many **sessions** (practice, qualifying, sprint, race).

- **season_entries** joins the three reference tables: each row ties one **driver** to one **constructor** for one **season** — this is the roster.
- One **session** has many rows in **results**, **qualifying**, **laps**, **pit_stops**, **stints**, **weather**, **live_timing**, and **predictions**. (sessions is the hub almost everything hangs off.)
- One **driver** appears in many **results**, **qualifying**, **laps**, **pit_stops**, **stints**, and **live_timing** rows.
- One **constructor** appears in many **results** and **qualifying** rows; one **status** classifies many **results**.
- **live_timing** is one row per driver per lap (the running order); **predictions** is one row per session per ~10-lap checkpoint, with the ranked grid inside its JSONB payload.